

all threads will die, even if that is not what you intended. Also, you will notice the type of the fourth argument in the `pthread_create` call, and the return type of the threads entry routine—whose prototype is `void * (*start_routine)(void *)`). These are `void*`, which means you can pass any type of data, and also return any type of data when using these functions.

Also, I highly recommend that you always check the return value of the functions. The return value on success for most `pthread` calls is 0, and a non-zero return value means some error has occurred; in which case, you should take appropriate action.

I would suggest you read the manual page (*man pthreads*) for general information and more details on the functions.

Mutex and condition variables

A common problem arises when you are accessing (say reading and writing) global data shared between multiple threads. This can cause an incorrect output. This is called race conditions, and must be avoided. A classic way to solve this is shown in the pseudo-code fragment below. You can use a mutex, which is a basic form of synchronisation, as follows:

```
lock_the_mutex(&mymutex)
    critical region
unlock_the_mutex(&mymutex)
```

The critical region is one where you are accessing or updating the shared data.

The POSIX data type for a mutex is `thread_mutex_t`. It can be statically or dynamically allocated. I have used '&' since the function requires that we pass the address of the mutex. In actual code, you need to use functions like `int pthread_mutex_lock(pthread_mutex_t *mutex)` and `int pthread_mutex_unlock(pthread_mutex_t *mutex)`. The mutex here allows you to ensure that at any instant, only one thread will be able to get the lock on the mutex atomically, and hence the other threads get blocked, since the mutex is now locked. Once the first thread releases the mutex, only then can another thread get the lock and proceed.

While you are using mutexes, please be cautious of deadlocks, a thread relocating a mutex (without unlocking), and similar problems. Let us look at a code fragment to illustrate one of the problems mentioned:

```
Thread1 {
    lock_the_mutex (&mutex_a);
    lock_the_mutex (&mutex_b);
}

Thread2 {
    lock_the_mutex (&mutex_b);
    lock_the_mutex (&mutex_a);
}
```

Note the reversed order of locking the mutexes. Suppose

both the threads complete the first step at almost the same time, what would happen? A deadlock—no thread will be able to proceed, as each is waiting to acquire a lock that the other holds. To solve this type of problem, there is the standard locking hierarchy—i.e., all programs that need `mutex_a` and `mutex_b` must always lock `mutex_a` first and then `mutex_b`. You can also first try to lock all the mutexes, and if any one of the set fails, release all previously locked mutexes. For this, you need to use a function like `int pthread_mutex_trylock(pthread_mutex_t *mutex)`. (You may download the file http://www.linuxforu.com/article_source_code/oct12/system_programming.zip to experiment with the mutex functions.)

Let me quickly introduce you to condition variables, which are also quite useful in certain cases. Now let's assume a thread needs to perform some action when a condition becomes true, such as, when it is waiting for some data to arrive in a queue, so that it can consume it. This queue is shared among multiple threads. Now you need a solution that is efficient and will reduce waiting or unnecessary polling. There is a mechanism by which one thread can signal another, which is waiting for some condition to be true, through a condition variable. Some simple pseudo-code below will help you understand the benefit:

```
thread1() {
    pthread_mutex_lock(&mutex);
    done++;
    pthread_cond_signal(&done_cond);
    pthread_mutex_unlock(&mutex);
}

thread2() {
    pthread_mutex_lock(&mutex);
    while (done == 0)
        pthread_cond_wait(&done_cond, &mutex);
    Take some action;
    pthread_mutex_unlock(&mutex);
}
```

In the above example, there are two threads, `thread1` and `thread2`. There is a condition variable, which is denoted by `done_cond` associated with a mutex variable. (A condition variable is always used in conjunction with a mutex.) `thread2` needs to wait till the condition becomes true (here, the condition is the change of the variable `done` to non-zero). Now this is shared between the threads. In actual cases, this can be a queue, list or anything. If, suppose, `thread2` is scheduled first, it gets a lock of the mutex, checks the condition `done==0` (the value of `done` is not changed since `thread1` is not scheduled), and then executes `pthread_cond_wait(&done_cond, &mutex)`. This function `pthread_cond_wait` puts `thread2` to sleep, while unlocking the mutex. So you do avoid polling here.

Now, when `thread1` gets a chance to run, it locks the mutex, changes the variable `done`, calls `pthread_cond_signal(&done_cond)` and unlocks the mutex. Since `thread1`